Networks and Systems Security 2

CSE 354/554

Instructor: Sambuddho

(Semester: Winter 2025)

Week 1–2: Jan 7

Introduction

- What is Security ?
- What is Computer Security ?
- What exactly is (are) insecure - software , hardware, users ?
- “Security and Networks don’t go together…” (-SMB)

Class Objective: What NSS 2 Would Teach You

- Designing networks, systems and everything related, keeping security in mind, through real world systems and tools.
  - Access controls [~3wks]
  - Software vulnerabilities and exploits [~2wks]
  - Basics of crypto-frameworks, message confidentiality, authentication [~3wks]
  - Network security protocols [~2.5wks]
  - Priavacy and anonymity [~2.5wks]
- How NSS 2 differs from NSS/SE/NS ?
  - NSS-1 : Focuses only on the fundamentals.
  - SE: Earlier avatar of NSS-1.
  - NS: Focus is complementary.

Grade Distribution and Administrativia

- **Grading:**
  - Assignments [30%] : 3 x 10%
  - Exercises [25%]: 4 x 6.25%
  - Project [30%]: 1 x 30%
  - Final exam [15%] : 1 x 15%
- **Textbook:**
  - *Network Security. PRIVATE Communication in a PUBLIC World. Charlie Kaufman, Radia Perlman and Mike Speciner. PHI Learning Pvt. Ltd. 2nd Edition, ASIN : B001ADIWNI, ISBN 978-81-203-2213-4*
  - *Computer Security: Principles and Practices (2nd Edition)*, W. Stallings and L. Browne, 2011, ISBN-13: 978-0132775069 (Selected Topics)
  - *Craft of Systems Security. J. Smith and J. Marchesini. ISBN-13: 978-0321434838*
  - *The Hacker Playbook 2: Practical Guide to Penetration Testing. Peter Kim. ISBN 13: 978151221456*
  - *The Hacker Playbook 3: Practical Guide to Penetration Testing. Peter Kim.* ISBN 13: 9781980901754

- **Contact email:** sambuddho@iiitd.ac.in
- **Mailing list:** cse554@iiitd.ac.in
- **Office Hours:** Friday. 12:00 PM - 1:00 PM (Room No. B503, New Academic Block)
- **TA office hours:** TBD

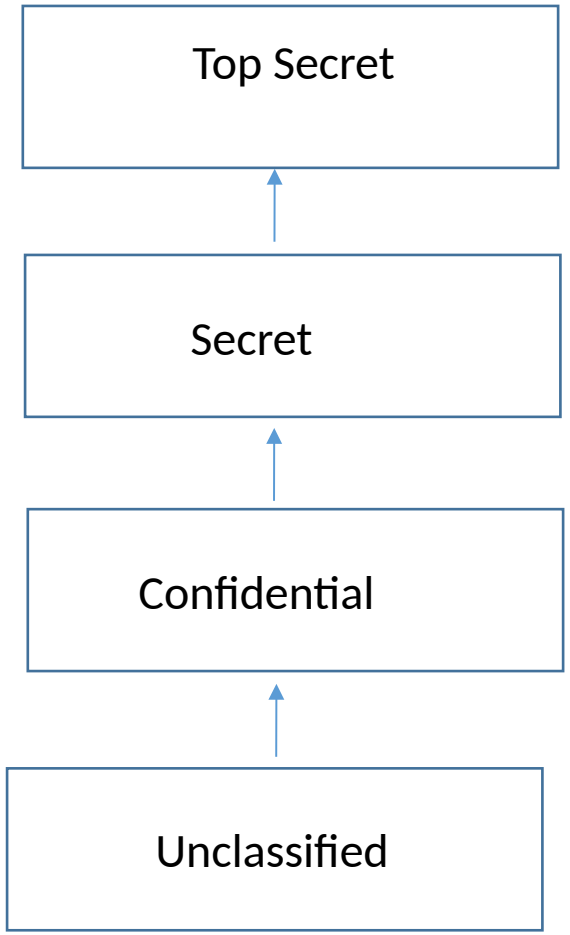
Traditional Access Control Models

- Access controls - DAC, MAC, RBAC and how they are implemented in popular OSes.
  - “Who” (aka *subject*) can access which resource (aka *object*): **Read, write, execute,** search, create, delete.
  - DAC: Owners of objects can modify their permissions.
  - MAC: System enforced rules based on access control hierarchy (MLS systems).
  - RBAC: Relies on a hierarchy of “roles” *e.g. Employee, Employer, Visitor etc.*
  - ACLs in modern OSes.
  - Access control matrix - describes who has access to which resource.

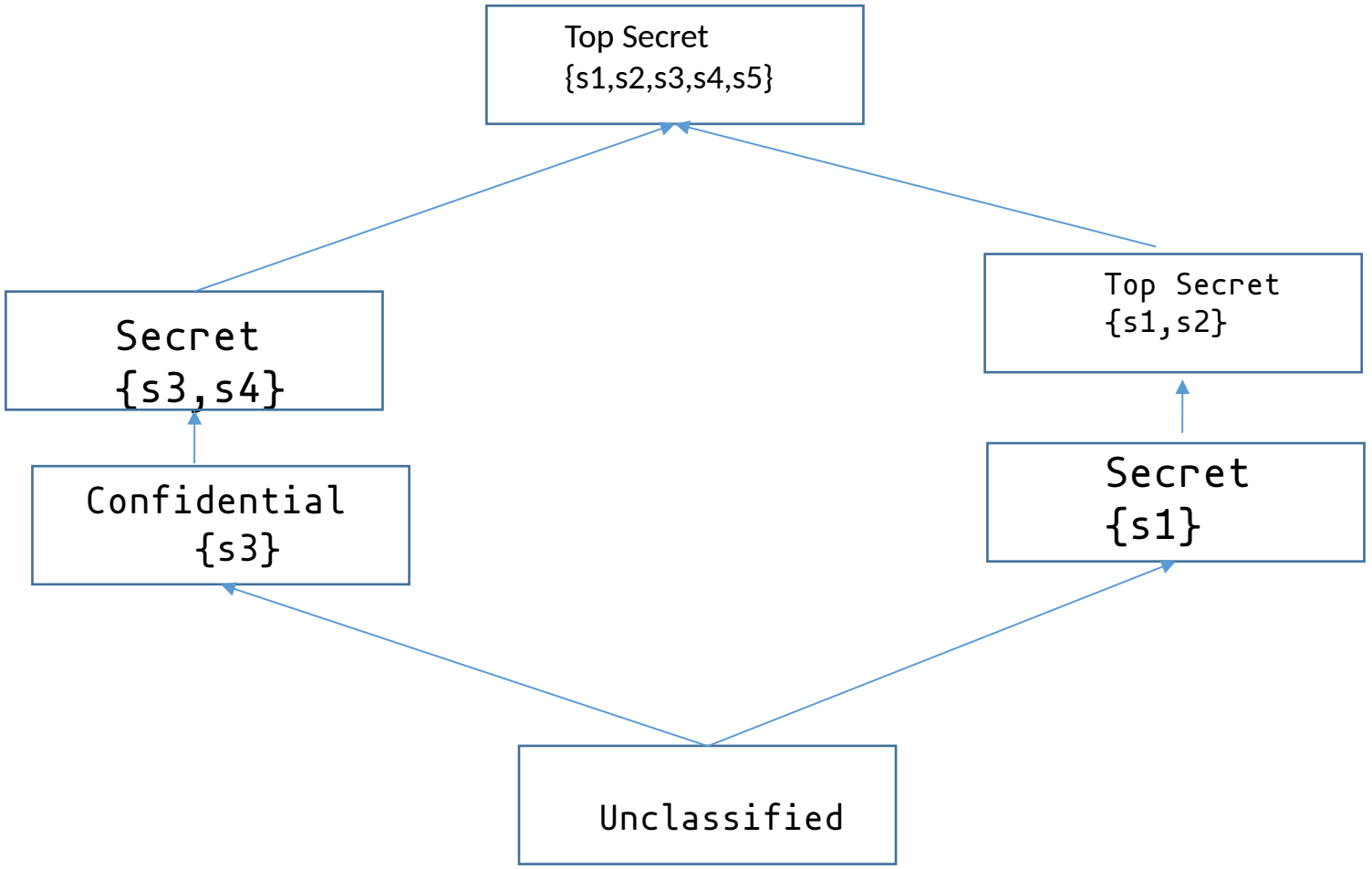
file1	Read	Write	Execute
Manager	Yes	Yes	Yes
Employee	Yes	No	Yes
Visitor	Yes	No	No

Security Models: Basics

Set ClearenceLevel = {Unclassified, Confidential, Secret, Top Secret}



Security Models: MLS (Multilevel Security)



- Subject set = {S1,S2,S3,S4,S5}
- Subjects are organized through
  hierarchies or levels of security
  E.g. {S1,S2} subset of {S1,S2,S3} and so has lesser security clearance compared to the latter.
- Is {S2,S3} less or more privileged than {S1,S4} ?

Bell-LaPadula Model

- Formalizes subjects and objects in clearance/category classes.

3 main principles:

The Simple Security Property: A subject can read an object only if the class of the subject dominates or equals the class of the object; aka *no-read-up rule:* A subject cannot read data from an object "above" it in the lattice.

The ⋆-Property (aka no-write-down rule): If a subject has simultaneous read access to object O1 and write access to O2 then O2 ≥ O1. Prevents (un)intended "leakage" of sensitive information.

Discretionary Security Property: S operates on O iff permitted by the access control matrix.

The Biba Model

- “Reversed” Bell–LaPadula Model: S and O are organized as per the integrity levels (unlike Bell LaPadula which is organized as per Confidentiality).
  - Prevent leakage of information from objects of higher integrity level to subjects of lower levels of integrity.
  - Prevent contamination of objects of higher integrity level from being written to by subjects (actions) with lower levels of integrity.

  - *Simple Integrity Axiom: No-read-down*
  - *The *Integrity Axiom: No-write-up*
  - *Invocation property: A subject can access request to objects at level equal or below.*

Contemporary Relevance of Classical Models

- Is Bell–LaPadula/Biba model relevant for modern systems ? (Yes/No/Maybe)
  - Yes/Maybe: Novel approach to model security of systems as subjects and objects and ensures some form of confidentiality across various *clearance–levels.*
  - No:
    - Modern systems' security is rooted in *interface security.*
    - *Very often* attacked because programs do not correctly check the boundary conditions.
    - Running code of on level at another level's security - running your cod with my privilege levels.
    - Too Unwieldy
      - How to maintain flow in only one direction for complex *lattice* may break OS enforced barriers - e.g. page boundaries of different processes having different permissions.
      - Manageability - Grouping users into categories and labelling them, adding removing new users to existing group(s).
      - Doesn't fit with various modern system designs *e.g.* Microkernels, embedded/mobile/low resource OSes designed keeping performance in mind (even Monolithic ones…).
      - Does not consider modern security concerns like Privacy/Anonymity etc.
      - *Trusthworthy computing:* Has different connotations wrt modern computing world.

Saltzer and Schroeder Model (KISS model)

- Contemporaneous to Bell-LaPadula and Biba Model
  - Economy of mechanism: Keeping design simple (aka KISS model)
  - Fail-safe defaults: Default action of the system that should be to deny access to someone or something until it has been explicitly granted the necessary privileges. Unhandled error could leave the system in an insecure state.
  - Complete mediation: Every object access needs to be authorized.
  - Open design: Obscurity doesn't lead to security (*e.g.* MS has been criticized in the past for closed designs that were ultimately insecure)
  - Separation of privileges
  - Least privileges: A user/process should operate with the smallest set of privileges necessary to complete the given task.
  - Least-common mechanism: Least possible sharing of information or resource between users. Could leak to inadvertent flow of information (Confinement problem).

Role of Underlying Systems and Platforms

- What policies/filters/mechanisms do underlying system platforms enforce or aid for implementing various forms of access controls?
  - Memory protection modes – paged virtual memory and segmentation
  - CPU privilege levels

    x86's 4 privilege level – most modern OSes operate only at 2 levels 0 (most privileged) and 3 (least privileged).

    Privileged operations: I/O,

    manipulate timer and H/W devices,

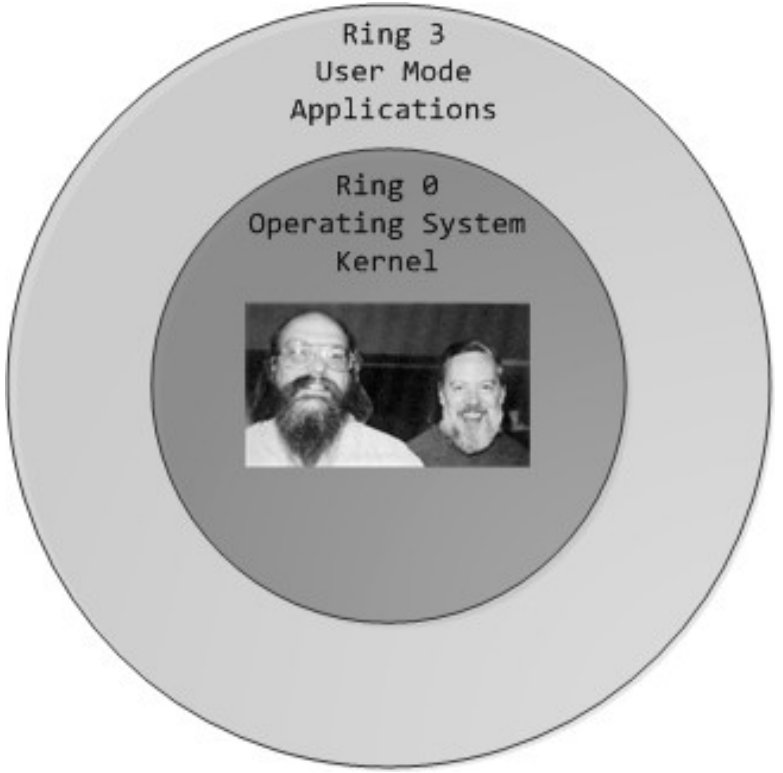
    MMU operations etc.

    Unprivileged operations: Ordinary operations like

    R/M operations, arithmetic/floating point ops. e*tc.*

    Advantage:

    Principle of (least) privileges



Network Based Access Control Mechanism

- Firewalls
- Proxies
- Host based approaches

Basic *nix Access Controls

- Every entity in a *nix system is a file – file, directory, I/O device interfaces, pipes, sockets *etc.* – each of these have access controls
- Used to implement DAC.
- Representation:

  10 bits – 9 for {u,g,o} x {r,w,x} + 1 directory or file (or character device, block device, named pipe, links (symbolic or hard links))

  ```
  -rw-rw-rw-    alice  alice  1.5M 2015-01-01  00:01  img1.jpeg
  drwxr-xr-x    alice  sys    4.0K 2014-01-01  00:01  mydirectory
  ```

  3 more bits – **setuid, setgid**, sticky

The /etc/passwd, /etc/shadows and /etc/groups Files

/etc/shadow files

alice:Ep6mckr0LChF:100063:0:7:7:1992873

1. Username
2. Encrypted password of the user. Users with passwd as ‘!’ or ‘*’ (meaning ?). Other possible values are blanks (meaning ?).
3. Date of last password change (expressed in the no. of days since Jan 1, 1970).
4. Min. password age = min. no. of days the users has to wait to change the password again. Could be 0 (no min.)
5. Max password age = The max. no. of days after which the user will have to change her password.
6. Password warning period = The number of days before a password is going to during which the user should be warned.
7. Password inactivity period = The number of days after a password has expired during which the password should still be accepted.
8. Account expiration date = Date of expiration of the account, expressed in no. of days since Jan 1, 1970

/etc/groups:

Shows group associations.

The /etc/passwd, /etc/shadows and /etc/groups Files

/etc/passwd file format:

alice:x:561:561:Alice Wonderlady:/home/alice:/bin/bash

1. Username
2. Password encrypted and stored in /etc/shadow file
3. UserID
4. GroupID
5. Real name associated the username
6. Home directory
7. Shell program associated with the user

How can you create a second “‘root”’ ?

File Permission Bits – Mode and Mask

- Octal representation

  0644 [(u:110 , rw-) , (g:100,r--) , (o:100,r--)]

- Files are by default created with a certain ‘mode’. The ‘mode’ is bitwise-ANDed to the inverted ‘umask’ bits.

  Default mode: 0666

  Default mask: 022

  Default permissions: 0644
- Changing the mask value: umask
- Changing system wide mask ?

The **setuid** and **setgid** Bits

- Turning on the bits: chmod u+s <filename>, chmod g+s <filename>
- Setuid/setgid bit:

  -When someone access your file, (s)he acquires your (user) privileges.

  - User A can access B's files as if it were B.

  - User in group A access files in group B, as if it were indeed part of group B.

  - Why can't we simply use permissions ?

  - A can access which B allows but not that which B ONLY can access.

  - Finer control: setuid(), seteuid(), setresuid()

  (similar controls using setgid(), setegid(), setresgid())

  - E.g. 'mount', 'ping', 'su', 'passwd'.

  - Can be useful but can lead to very dangerous security vulns.

Safe Way to Use setuid() (and Friends)

- Safe ways to use setuid() functions:
  - User A's process access user B's programs – performs the necessary execution as user B
  - User B's program performs the privileged operation and then goes back to user A's permissions.
  - Principle of least privileges

A's program --> B's program

```
{
   unprivileged operations;
   real_uid = getuid(); //get calling processes uid
    privileged operations;
  seteuid(real_uid); //relinquish privileges
}
```

Dangerous Ways of Using setuid() Programs

- Not relinquishing privileges after using them.
  - Forks(), execv() (and friends), signal() handlers may not reset privileges.
  - Some programs are associated with ‘pipes’ and ‘character devices’ associated to various system critical resources – *e.g.* /dev/kmem associated with kernel memory. ‘kmem’ associated users – programs associated with ‘kmem’ user (owner) could crash the program through some debugger and access kernel memory.
  - CVE-2001-1384: Vulnerability in ptrace() program. exec() used. It is possible for a traced process to exec() a setuid image if the tracing process is setuid.

Allied Functions

- getuid(): Obtain the UID of the process which is running the suid program.
- geteuid(): UID with which the suid program is running.
- getresuid(): Obtain real, saved and effective UID.

Why do you need the saved UID ?

Allows process to resume privileges.

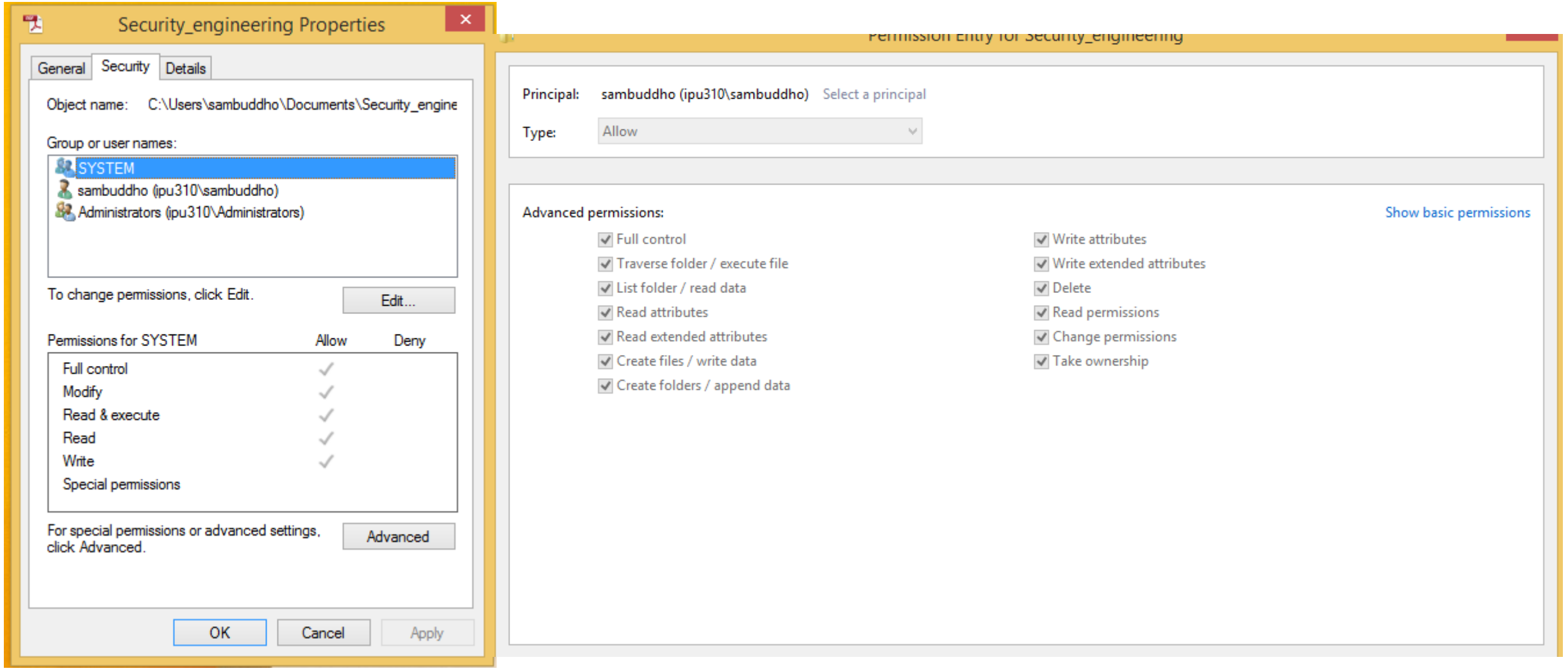
- Start as root.
- Drop to real UID after operations are done
- Need to resume root ? You cannot get it back…
- You need saved UID. (Doesn’t work for ‘root’, in Linux atleast)

Access Control Features in Windows 7/2008 server/8/10/11

- Windows model differs slightly from *nix model:
  - Objects (I/O devices, files, directories).
  - Subjects (users and programs)

  - Access Tokens: Similar to default DAC permissions for users in *nix + /etc/passwd + /etc/shadow + /etc/group info.
  - DACLs and ACEs:
    - DACLs - another ‘cool’ way of referring to ACLs
  - Permissions: r,w,x,m,d,chown
    - Security descriptor - created within a file - stores DACLs and ACEs.
    - Explicit vs inherited.
    - Nothing is ‘world accessible’, unlike *nix; however these is a ‘local access group’ which approximates the former.

Access Control Features in Windows 7/2008 Server/8/10/11



Access Control Lists in Linux

- Often not turned on by default.
- What you can achieve with standard permissions could also be used with ACLs.
- ‘setfacl’, ‘getfacl’
- Works with the regular permissions (but could also potentially break it!).
- There are multiple users – owner and named users, owning group and named group and others.
- Which one is trusted – the owner or the named users ?
  - ‘mask’ bits – different from ‘umask’

Access Control Lists in Linux

- Turning on ACL
  - Edit /etc/fstab

  /dev/hda2 ext4 **acl**,errors=remount-ro 0 1
  - Remount

  root@hostname:~# mount -o remount /

  root@hostname:~# mount | grep root

  /dev/hda2 on / type ext4 (rw,**acl**,errors=remount-ro)
  - Application layer utilities

  root@hostname:~# apt-get install acl

‘Set’ and ‘Get’ permissions

root@hostname# setfacl -m g:testusers:r appdir/
-m: specify/’set’ permission

root@hostname# getfacl appdir/
file: appdir/
owner: root
group: appgroup
user::rwx
group::rwx
group:testusers:r–
mask::rwx
other::r-x

‘Set’ permissions recursively

- root@testvm:/var/tmp# setfacl -Rm g:testusers:r appdir/
  root@testvm:/var/tmp# getfacl appdir/*

```
  # file: appdir/appuser1
  # owner: appuser1
  # group: appgroup
user::rwx
group::r-x group:testusers:r-- mask::r-x
other::r-x
  # file: appdir/appuser2
  # owner: appuser2
  # group: appgroup
 user::rwx
 group::r-x group:testusers:r-- mask::r-x
 other::r-x
```

Default ACLs

Default value of ACL applied to all files and subdirectories created within a directory.

```
root@host:/var/tmp# setfacl -dm g:testusers:r appdir/
root@host:/var/tmp# touch appdir/file1
root@host:/var/tmp# mkdir appdir/dir1
root@host:/var/tmp# getfacl appdir/*
# file: appdir/file1
# owner: appuser1
# group: appgroup
user::rwx
group::r-x
group:testusers:r-- mask::r-x
other::r-x
```

```
# file: appdir/dir1
# owner: root
# group: root
user::rwx
group::rwx
group:testusers:r-- mask::rwx
other::r-x
default:user::rwx
default:group::rwx
default:group:testusers:r-
```

Removing ACLs

-b: Removes all ACLs from a directory (go back to regular permissions)

-x: Specific rules are removed

```
root@host:/var/tmp# setfacl -x u:testusers:x appdir/ root@host:/var/tmp# getfacl appdir/
# file: appdir/
# owner: root
 # group: appgroup
user::rwx
group::rwx
group:testusers:r-
mask::rwx
 other::r-x
default:user::rwx
default:group::rwx
default:group:testusers:r- x
default:rwx
default:other::r-x
```

Access Control Lists in Linux

Type	Text Form
owner	user::rwx
named user	user:name:rwx
owning group	group::rwx
named group	group:name:rwx
mask	mask::rw-
other	other::rwx

Entry Type	Text Form	Permissions
named user	user:geeko:r-x	r-x
mask	mask::rw-	rw-
	effective permissions:	r--

Access Control Lists in Linux – Directories and Files

- Directories have ‘default’ permissions – ‘default:user’, ‘default:group’ and ‘default:other’. Subdirectories inherit those permissions and default permissions

- Files inherit them as regular permissions.
  - Files are created with mode of 0666. Whatever permissions is not granted in the mode are removed from the ACL permissions granted to the file.
  - *Doesn’t really echo with the default definition.*

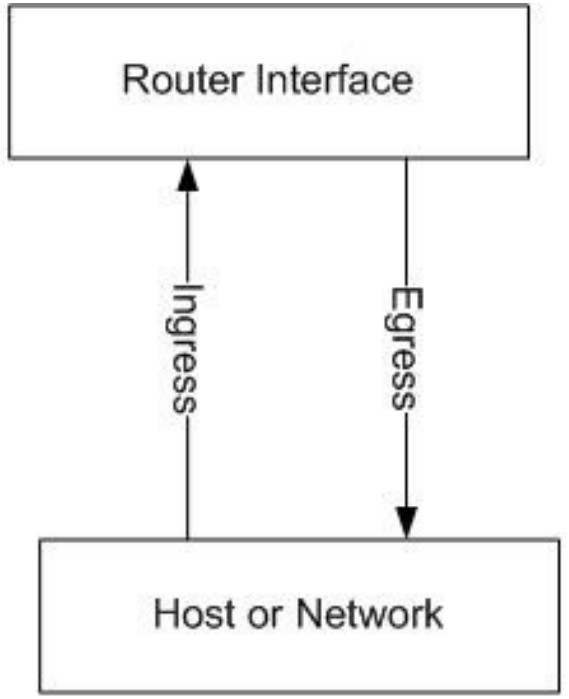
- Possible vulnerabilities when using ACLs ? NFSv2 and v3 FS vulns. Reported in Linux Kernel 2.6.14 and 2.6.15: Failure to correctly validate privileges remotely. **CVE-2005-3623**

Firewalls: Network Access Controls

Access control lists in routers:

- “Mimics” the behavior of ACLs in filesystems.

```
permit icmp any any
permit ip 20.0.0.0/8 10.0.0.0/8
permit tcp any 80
deny ipv6 any any
deny ip any any
```



Match order – *first match / last match*

Advantages and Disadvantages of ACL Filters

Advantages:

- Easy to implement – works by inspecting header fields (IP/TCP/UDP fields).
- Efficient compared to full blown firewalls.
- Works well for customized access control to specific network segments.

Disadvantages:

- Stateless (most implementations)

Stateless vs Statefull Firewalling

Stateless –

- Involves filtering individual packets based on their header fields.
- Each individual packet may not be correlated to its previous or consequent packets.

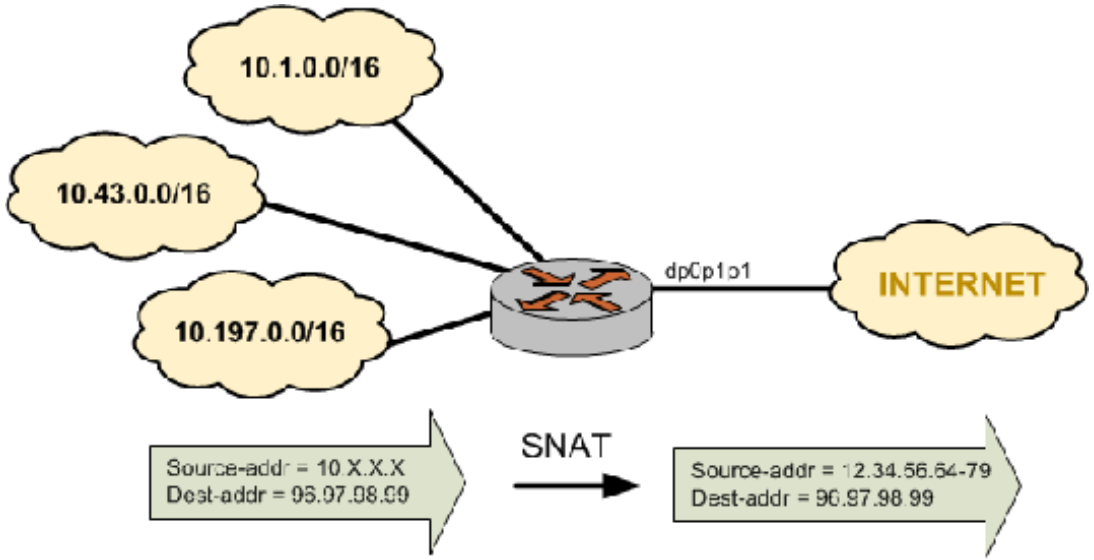
Statefull –

- Keeps track of individual connection states.
- Can be used to filter packets of different individual connections – connection related to one known connection – aka connection tracking.

Functions of Traditional Firewalls

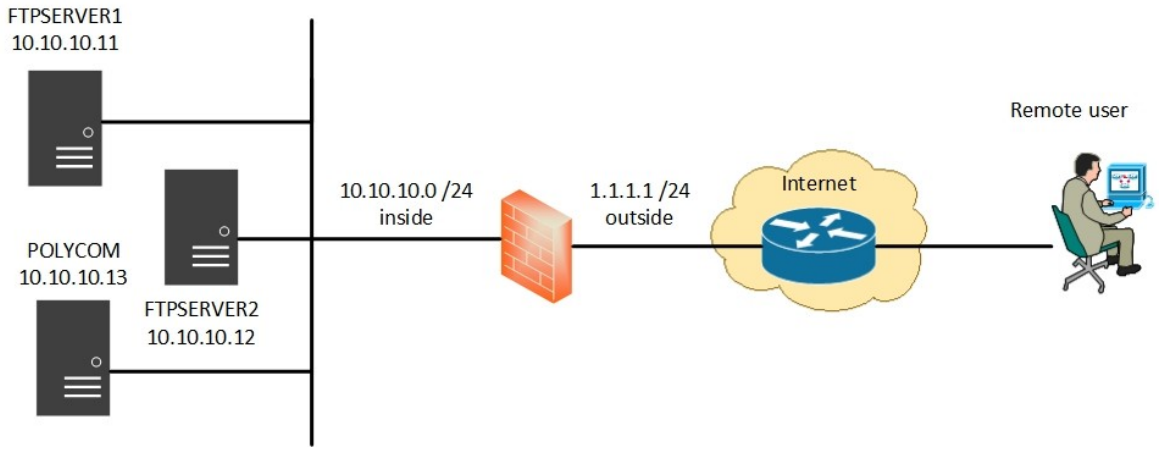
- Packet filtering based on header information.
- Connection tracking – statefull firewalling e.g. FTP session.
- SNAT/DNAT
  - Network address translation – preserve IP addresses. Internal IP address ranges different from the outside ones.
  - Internal IP addresses wouldn’t clash with outer IP addresses
- VPN gateway
  - Virtual Private Network
  - Different modes of communication
  - Packet encapsulation, encryption (confidentiality) and user authentication.
  - Gateway manages it.
- Authentication
  - Authenticating and authorizing internal and external users.

SNAT and DNAT



Source NAT: Many clients share the same outbound connection

Destination NAT: Client doesn't get to see the actual IP address of the service.



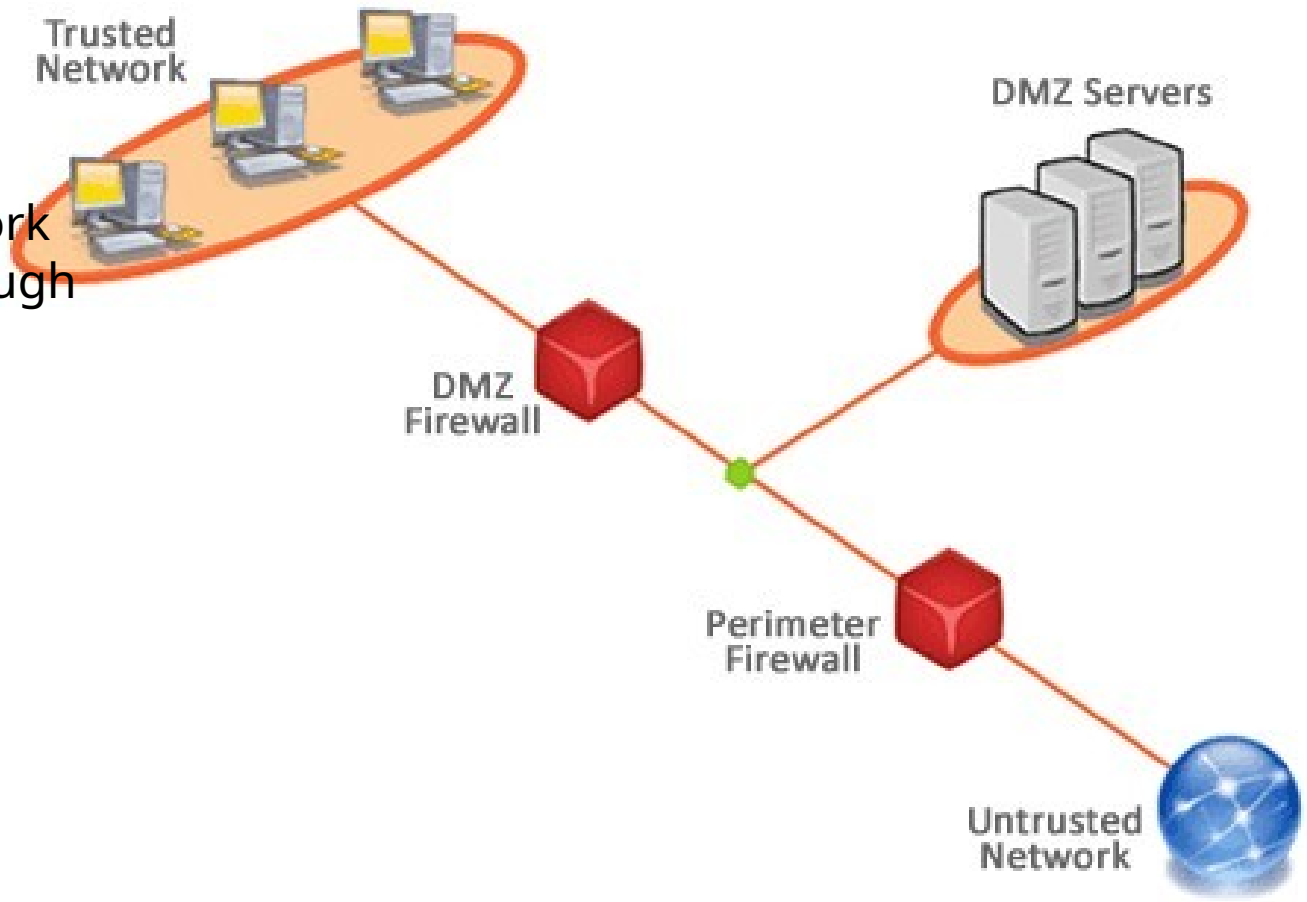
De-Militarized Zone (DMZ)

Trusted network:
- Never exposed to the outside world.
  (*e.g.* organizational LAN)
- DMZ firewall protects the trusted network
  trusted network; access the world through
  SNAT (at the DMZ firewall).

DMZ firewall: Protect the trusted network

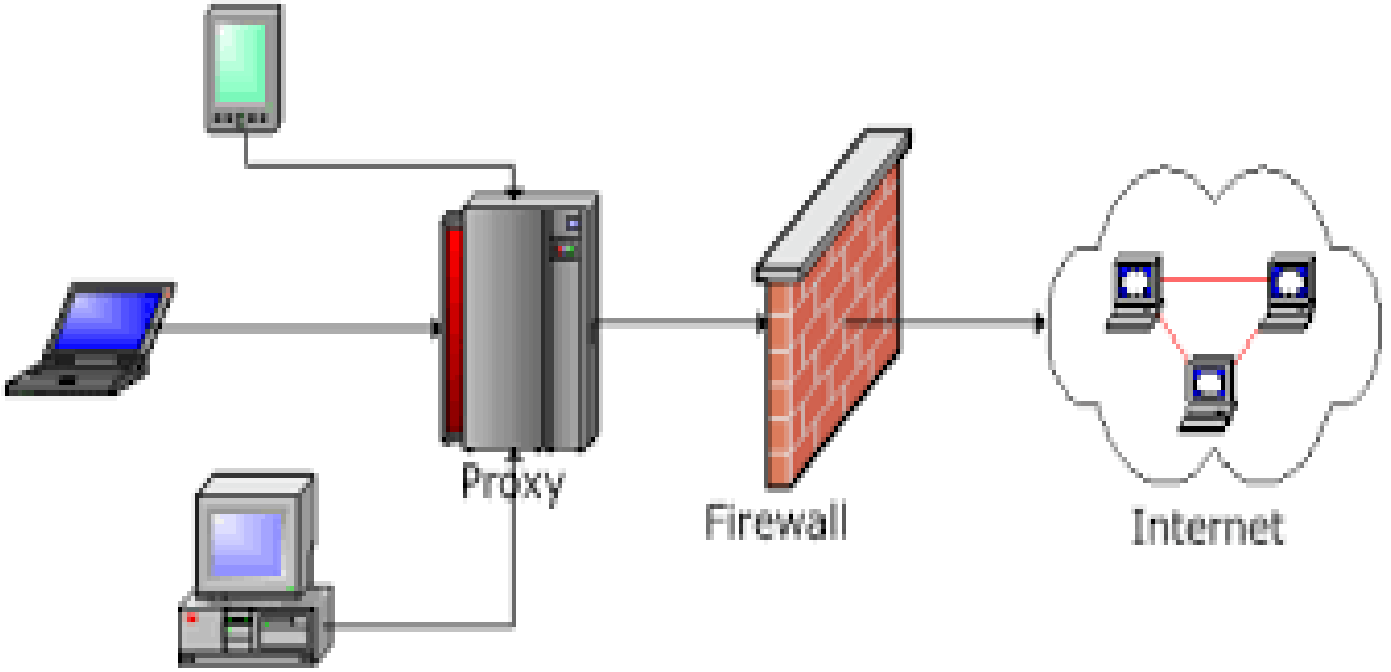
DMZ servers: Visible to the outside world

Perimeter firewall: Fw rules for the entire organization

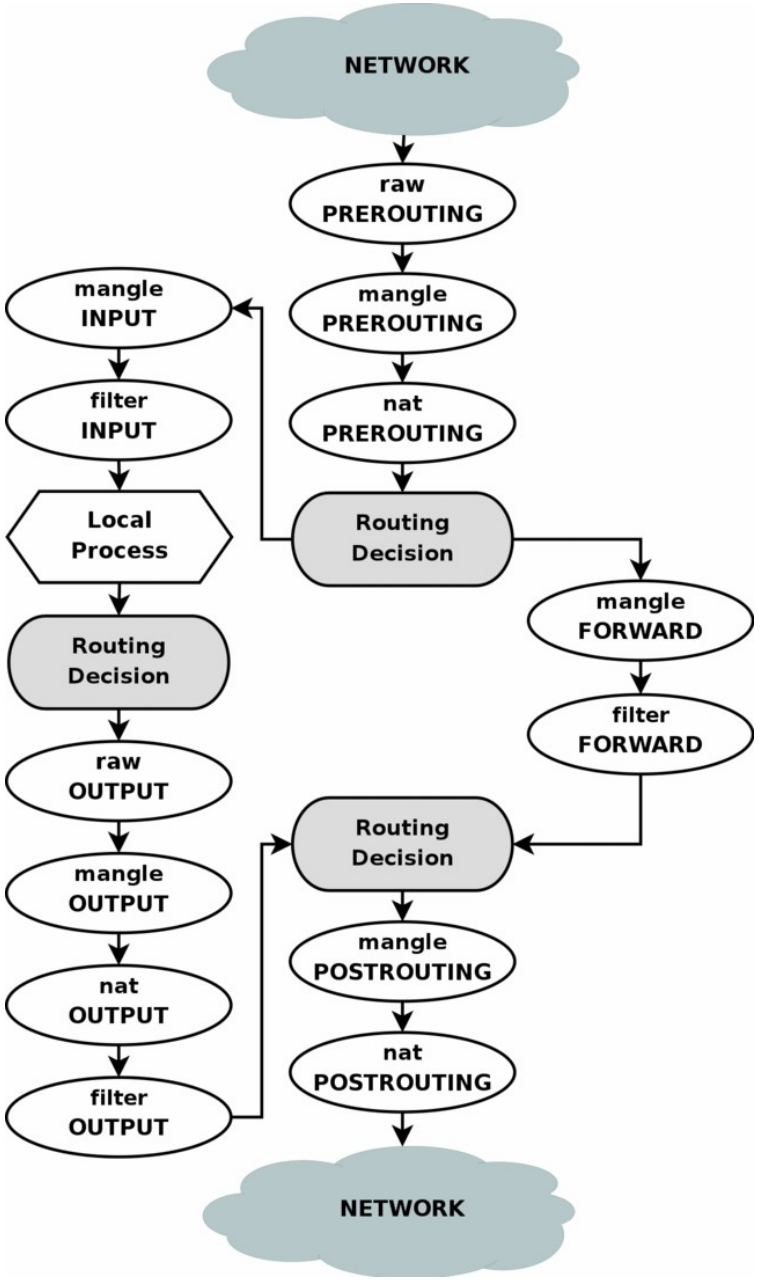


Proxy Based Filtering

- How are proxies different from Firewalls ?
- Traditionally proxies work at L5 while firewalls work at L3.
- Proxies allow additional complex filter capabilities.
- Unlike proxies don’t require configuration at the client. However trans-proxies work just the same way.



Case Study: Linux Netfilter/IPTables

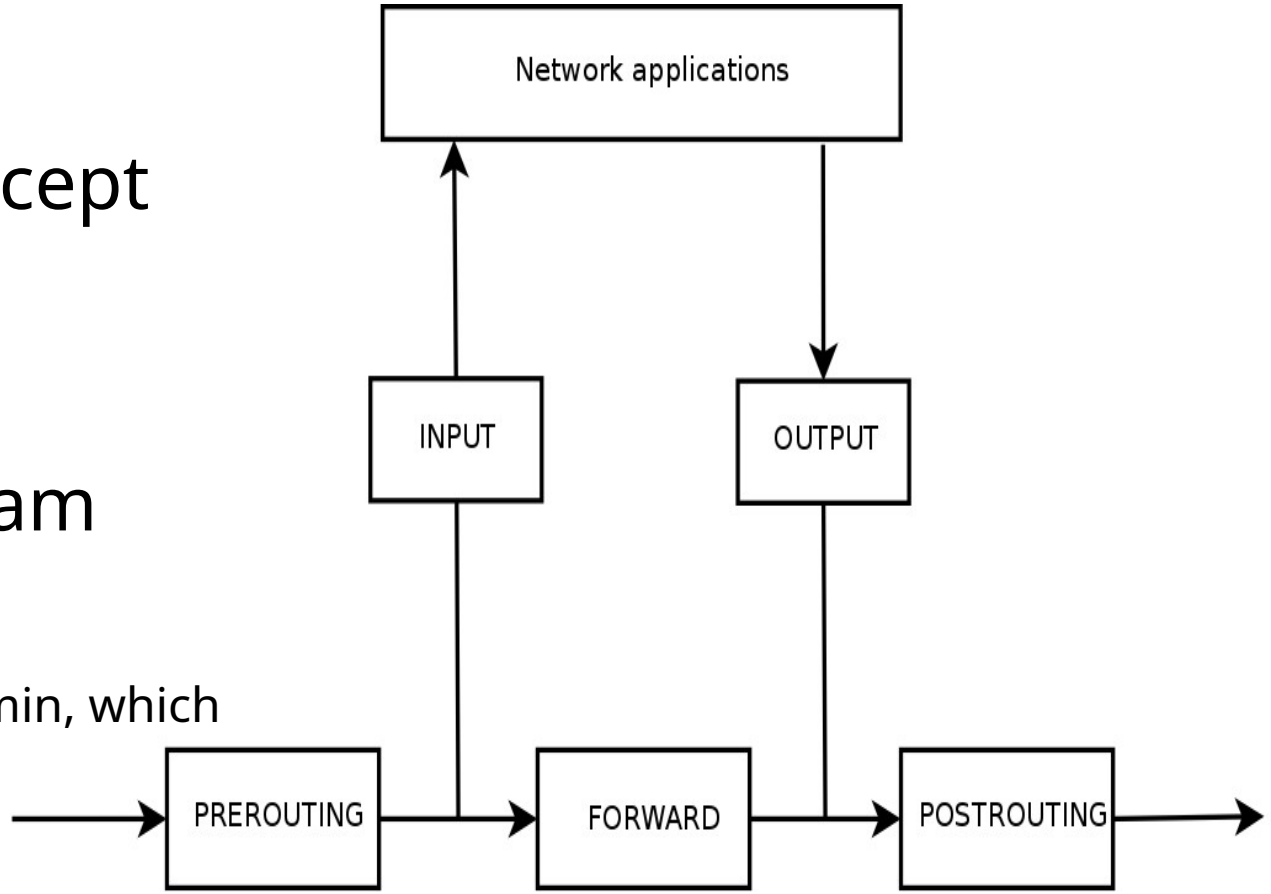


Case Study: Linux Netfilter/IPTables

- Kernel based firewall framework consisting of set of call-back functions that can be used to intercept individual packets.

- **Iptables** – Linux appmode program which uses the netfilter hooks.

(derived from ipchains, which is derived from ipfwadmin, which Is derived from ipfw of FreeBSD)



Linux Netfilter/IPTables: Hook Functions.

NF_IP_PRE_ROUTING – First function to be called, regardless of where the packet is destined to. Also used for DNAT

NF_IP_LOCAL_IN – Called only for packets going to the said host.

NF_IP_FORWARD – Called for packets being forwarded to another host.

NF_IP_LOCAL_OUT – Called for traffic being sent out (originating at the said host).

NF_IP_POST_ROUTING – Last hook function to be called. Also used for SNAT.

Linux Netfilter/IPTables: Hook Functions.

What to do after intercepting packets –

- NF_ACCEPT: continue traversal as normal.
- NF_DROP: drop the packet; don't continue traversal.
- NF_STOLEN: I've taken over the packet; don't continue traversal.
- NF_QUEUE: queue the packet (usually for userspace handling).
- NF_REPEAT: call this hook again.

Linux Netfilter/IPTables: IPTables Examples.

#iptables -L -n –v

#list existing rules

#iptables -I INPUT 2 -s 202.54.1.2 -j DROP

#Append to rules no. 2 for INPUT hook.

#iptables -A INPUT -p tcp --dport 80 -j DROP
#iptables -A INPUT -i eth1 -p tcp --dport 80 -j DROP

#iptables -A INPUT -p tcp -s 1.2.3.4 --dport 80 -j DROP
#iptables -A INPUT -i eth1 -p tcp -s 192.168.1.0/24 --dport 80 -j DROP

#iptables -A INPUT -p icmp --icmp-type echo-request -j DROP

#iptables -A INPUT -m mac --mac-source 00:11:2f:8f:f8:f8 -j DROP

#MAC address filtering

Nftables: Sucessor to Iptables

- Issues/limitations of Iptables:
  - Mostly involved with matching rules to packets with no regards to semantics (either at the application or kernel layer).
  - Non-atomic rule addition and removal.
  - More rules --> linearly increasing latency.
  - One rule, one match, one function. Matching multiple packets and processing multiple rules requires external tools (e.g. ipset).
  - No kernel notification if rules are added or removed.

Nftables: Basics

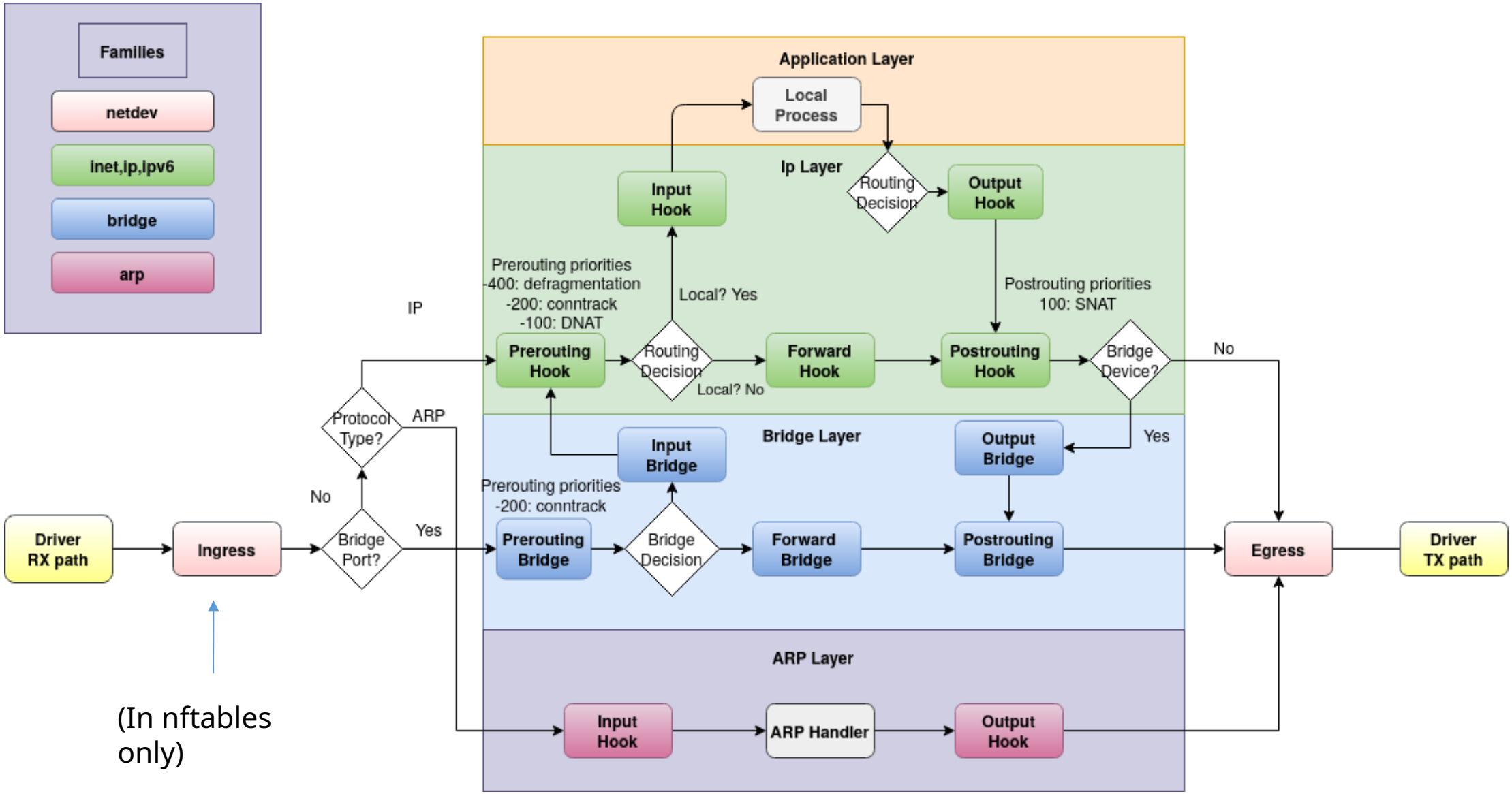
- Some advantages over iptables:
  - Supports sets and maps for non-linear rule matching, rule compaction etc.
  - Atomic rule addition and removal.
  - Every rule has a unique rule identifier, making for easy rule management.
  - Kernel rule notification.
  - No match --> action. Everything is an expression to implement functionality.
  - Rule handling, semantic checking etc. in user space; packet processing in kernel space.
  - Multiple actions per rule.

- Rule creation and usage example:

  nft add rule ip filter input tcp dport { 22 } accept

```
#!/usr/sbin/nft -f
# Flush the rule set flush ruleset table inet example_table { chain example_chain {
# Chain for incoming packets that drops all packets that
# are not explicitly allowed by any rule in this chain
type filter hook input priority 0; policy drop; # Accept connections to port 22 (ssh)
tcp dport ssh accept } }
```

Nftables Netfilter Integration



Nftables: Usage

- Shows fw rules:

  # nft list ruleset

- Create tables. Tables hold chains which hold collection of rules (rulesets) but no named built-in chains like iptables.:

#nft (add | create) chain [<family>] <table> <name> [{
<type> hook <hook> [device <device>] priority <priority> \; [policy <policy> \;] }]
#nft (delete | list | flush) chain [<family>] <table> <name>
#nft rename chain [<family>] <table> <name> <newname>

Family: ip, ip6, inet, arp, bridge

Type: filter, route, NAT

Hook: [ip, ip6] : prerouting, input, forward, output, postrouting.

Priority: refers to a number used to order the chains or to set them between some Netfilter operations. Possible values are: NF_IP_PRI_CONNTRACK_DEFRAG (-400), NF_IP_PRI_RAW (-300), NF_IP_PRI_SELINUX_FIRST (-225), NF_IP_PRI_CONNTRACK (-200), NF_IP_PRI_MANGLE (-150), NF_IP_PRI_NAT_DST (-100), NF_IP_PRI_FILTER (0), NF_IP_PRI_SECURITY (50), NF_IP_PRI_NAT_SRC (100), NF_IP_PRI_SELINUX_LAST (225), NF_IP_PRI_CONNTRACK_HELPER (300).

Nftables: Sample Fw Rules

#nft (add | create) chain [<family>] <table> <name> [{
 <type> hook <hook> [device <device>] priority <priority> \; [policy <policy> \;] }]
#nft (delete | list | flush) chain [<family>] <table> <name>
#nft rename chain [<family>] <table> <name> <newname>

Basic routing firewall

root #sysctl -w net.ipv4.ip_forward = 1

#!/sbin/nft -f

flush ruleset

table ip filter {

allow all packets sent by the firewall machine itself c

chain output { type filter hook output priority 100; policy accept; } # allow LAN to firewall, disallow WAN to firewall

chain input { type filter hook input priority 0; policy accept; iifname "lan0" accept iifname "wan0" drop } # allow packets from LAN to WAN, and WAN to LAN if LAN initiated the connection

chain forward { type filter hook forward priority 0; policy drop; iifname "lan0" oifname "wan0" accept iifname "wan0" oifname "lan0" ct state related,established accept }

}

Nftables: Sample Fw Rules

Typical IPv4 Workstation

```
#!/sbin/nft -f
flush ruleset
# ----- IPv4 -----
table ip filter{
chain input { type filter hook input priority 0; policy drop;
        ct state invalid counter drop comment "early drop of invalid packets“
        ct state {established, related} counter accept comment "accept all connections related to connections made by us"
        iif lo accept comment "accept loopback"
        iif != lo ip daddr 127.0.0.1/8 counter drop comment "drop connections to loopback not coming from loopback"
        ip protocol icmp counter accept comment "accept all ICMP types"
        tcp dport 22 counter accept comment "accept SSH"
        counter comment "count dropped packets" }
chain forward{ type filter hook forward priority 0; policy drop;
        counter comment "count dropped packets"     }
# If you're not counting packets, this chain can be omitted.
chain output {type filter hook output priority 0; policy accept;
       counter comment "count accepted packets" } }
```

Case Study: FreeBSD pf (packetfilter)

- Originally in OpenBSD 3.0 --> Ported to FreeBSD 5.x
- Intgertated into the kernel (much like Linux netfilter)
- Functions: (statefeul) packet filter, NAT, connection tracking, redirection, MAC address filtering, traffic redirection, predefined filtering profiles (*anchors*), connection throttling/rate-limiting, bruteforce detection etc.
- pfsense

Much Like Linux/Netfilter

- pfil(9) : In-kernel framework to build custom firewalling modules (instead of full blown firewall).
- Heads: `Points' where the firewalling can happen.
- Separation between kernel framework semantics and application layer interface.

```
PFIL(9)                FreeBSD Kernel Developer's Manual                PFIL(9)

NAME
     pfil, pfil_head_register, pfil_head_unregister, pfil_link, pfil_run_hooks
     -- packet filter interface

SYNOPSIS
     #include <sys/param.h>
     #include <sys/mbuf.h>
     #include <net/pfil.h>

     pfil_head_t
     pfil_head_register(struct pfil_head_args *args);

     void
     pfil_head_unregister(struct pfil_head_t *head);

     pfil_hook_t
     pfil_add_hook(struct pfil_hook_args *);

     void
     pfil_remove_hook(pfil_hook_t);

     int
     pfil_link(struct pfil_link_args *args);

     int
     pfil_run_hooks(phil_head_t *, pfil_packet_t, struct ifnet *, int,
         struct inpcb *);
```

Heads are traversed

```
/* Argument structure used by packet filters to register themselves. */
struct pfil_hook_args {
        int              pa_version;
        int              pa_flags;
        enum pfil_types  pa_type;
        pfil_func_t      pa_func;
        void            *pa_ruleset;
        const char      *pa_modname;
        const char      *pa_rulname;
};
```

hook function

Case Study: FreeBSD pf (packetfilter)

- Enable:

```
# sysctl net.inet.ip.forwarding=1
# sysrc pf_enable=yes
# sysrc pflog_enable=yes

(in /etc/rc.conf)
pf_rules="/path/to/pf.conf"
pflog_logfile="/var/log/pflog"
gateway_enable="YES"
```

- Most basic rules: block all incoming packets; allow all outbound packets while mainting their state.

```
block in all
pass out all keep state
```

Case Study: FreeBSD pf (packetfilter)

- *Lists* and *macros*

```
tcp_services = "{ ssh, smtp, domain, www, pop3, auth, pop3s }"
udp_services = "{ domain }"
block all
pass out proto tcp to any port $tcp_services keep state
pass proto udp to any port $udp_services keep state
```

- Interface groupings

```
pass from xl1:network to any port $ports keep state


ext_if = "xl0"    # macro for external interface - use tun0 for PPPoE
int_if = "xl1"    # macro for internal interface
localnet = $int_if:network
# ext_if IP address could be dynamic, hence ($ext_if)
nat on $ext_if from $localnet to any -> ($ext_if)
block all
pass from { lo0, $localnet } to any keep state
```

Case Study: FreeBSD pf (packetfilter)

- Port grouping

```
client_out = "{ ftp-data, ftp, ssh, domain, pop3, auth, nntp, http, https, cvspserver, 2628, 5999, 8000, 8080 }"
pass inet proto tcp from $localnet to any port $client_out flags S/SA keep state
```

- Managing ICMP

```
pass inet proto icmp from $localnet to any keep state
block inet proto icmp from any to $ext_if keep state
```

Mandatory Access Controls in Linux: SELinux

- Discretionary Access Controls cannot solve all problems –
  - Owner sets access privileges for files.
  - Infeasible to set privileges of ALL files in the system.
  - No mediated access
  - Recall SUID vulnerabilities
  - Need a second line of defense if the DAC is compromised.
  - First line of defense DAC. If allowed by DAC then MAC is checked (enforced).
- What all can you do with SELinux –
  - Enforce MAC/RBAC.
  - Fine tuned access control, that acts as second line of defense.
    - Administrator can enforce which system calls a (or a certain class) of users can use.

Getting Started with SELinux

/etc/sysconfig/selinux , /etc/selinux/config

SELINUX=disabled

SELINUX=**permissive**

SELINUX=**enforcing**

sestatus

SELinux status: enabled

SELinuxfs mount: /sys/fs/selinux

SELinux root directory: /etc/selinux

Loaded policy name: targeted

Current mode: enforcing

Mode from config file: enforcing

Policy MLS status: enabled

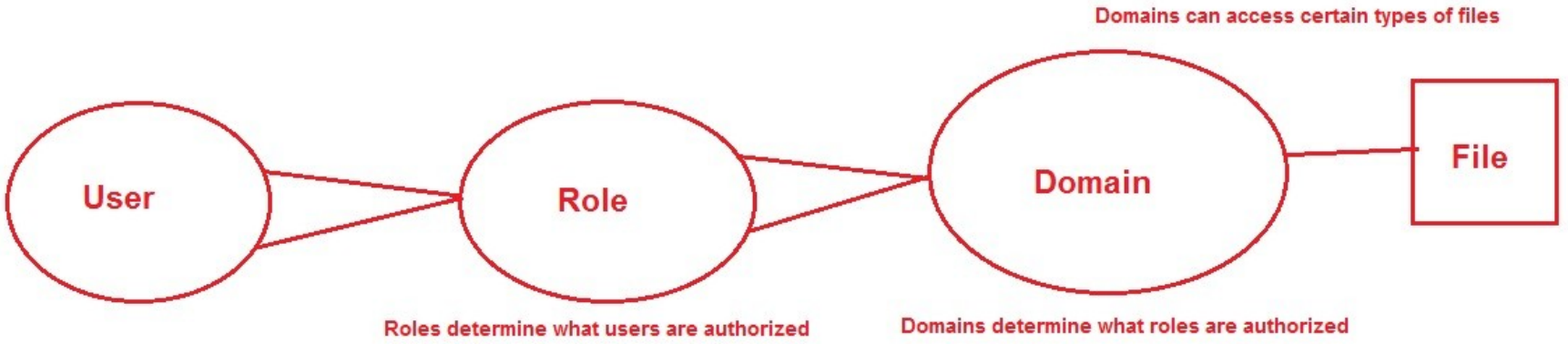
Policy deny_unknown status: allowed

Max kernel policy version: 28

- Three modes:
  - Enforcing : Policy enforcement
  - Permissive : Only warning but no policy enforcement
  - Disabled : Disabled
- Levels
  - Targeted : Enforces on specific processes
  - MLS : Multilevel security

Mandatory Access Controls in Linux: SELinux

- Policy basics
- **Users:** Basic set of pre-built users to which every user maps to.
- **Roles:** Gateway that sits between a user and a process. A role defines which users can access that process. Roles are not like groups, but more like filters: a user may enter or assume a role at any time provided the role grants it.
- **Subjects and objects:** Users and targeted objects.
- **Domains:** The context within which an SELinux subject (process) runs. A wrapper around the subject that dictates what it can/can't do.
- **Types:** Context for a file that stipulates its purposes – e.g. web page file or a config file in /etc/



SELinux Boolean Settings

semanage boolean -l | less

```
ftp_home_dir                   (off  ,  off)  Allow ftp to home dir
smartmon_3ware                 (off  ,  off)  Allow smartmon to 3ware
mpd_enable_homedirs            (off  ,  off)  Allow mpd to enable homedirs
xdm_sysadm_login               (off  ,  off)  Allow xdm to sysadm login
xen_use_nfs                    (off  ,  off)  Allow xen to use nfs
mozilla_read_content           (off  ,  off)  Allow mozilla to read content
ssh_chroot_rw_homedirs         (off  ,  off)  Allow ssh to chroot rw homedirs
mount_anyfile                  (on   ,   on)  Allow mount to anyfile
...
...
```

getsebool/setsebool: Change the SELinux booleans.

Mandatory Access Controls in Linux: SELinux

- File Security Contexts –

  ls -Z /etc/*.conf

  -rw-r- - r - - root root system_u:object_r:locale_t /etc/localte.conf
  -rw-r- - r - - root root system_u:object_r:etc_t /etc/man_db.conf

  system_u: mapping between user and SELinux user.
  object_r: SELinux role.
  etc_t: Type

Mandatory Access Controls in Linux: SELinux

Specific label assigned to a process by SELinux that inform about the rights and privileges being granted to a process.

Process contexts.

root #ps -eZ | grep auditd
system_u:system_r:auditd_t 24008 ? 00:00:00 auditd

root #ps -eZ | grep -E '(auditd|sshd|grep)'
system_u:system_r:auditd_t:s0 root 3934 ? 00:00:00 /sbin/auditd system_u:system_r:kernel_t:s0 root 3946 ? 00:00:00 [kauditd] system_u:system_r:sshd_t:s0-s0:c0.c1023 root 4159 ? 00:00:00 /usr/sbin/sshd unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 root 4240 ? 00:00:00 grep

Several process could be assigned to one context. One process is always assigned exactly one context.

Mandatory Access Controls in Linux: SELinux

- Context inheritance –
  - Unless otherwise specified the files and processs are created with contexts of their parent files or processes
  - For files: Copying to “out-of-context” locations results in loosing their original contexts. They acquire the contexts of the destinations.

- SELinux users –
  - Different from regular users – you cannot login with SELinux users.
  - There are only a small set of these users – *e.g.*
    - **guest_u**: This user doesn't have access to X-Window system (GUI) or networking and can't execute su / sudo command.
    - **user_u**: This user has more access than the guest accounts (GUI and networking), but can't switch users by running su or sudo.
    - **staff_u**: Same rights as user_u, except it can execute sudo command to have root privileges.
    - **system_u**: This user is meant for running system services and not to be mapped to regular user accounts.

  - Mapping between ordinary users and SELinux users: *Role* (defined through the above set of users)

SELinux Policy Type Enforcement

Specification for policy enforcement: How process access resources.

- allow <domain> <type>:<class> { <permissions> };

root #ps -eZ | grep auditd
system_u:system_r:auditd_t 24008 ? 00:00:00 auditd

root #sesearch --allow --source auditd_t --target auditd_log_t --class file --perm write
Found 1 semantic av rules:
allow auditd_t auditd_log_t : file { ioctl read write create getattr setattr lock append unlink link rename open } ;

Contoling File Contexts Yourself

root #semanage fcontext -l | grep auditd_log_t
/var/log/audit(/.*)? all files system_u:object_r:auditd_l

The **semanage** utility uses the files
in /etc/selinux/*/contexts/files as its source.

References

[BL76] D. Bell and L. Padula. *Secure Computer Systems: Unified Exposition and Multics Interpretation.* The MITRE Corp., March 1976. Technical Report ESD-TR-75-306.

[Bib77] K. Biba. *Integrity Considerations for Secure Computer Systems.* The MITRE Corp., April 1977. Technical Report ESD-TR-76-372.

[SS75] J. Saltzer and M. Schroeder. The Protection of Information in Computer Systems. *Proceedings of the IEEE 63(9): 1278 – 1308, 1975*